

740. Delete and Earn

Code:

```
class Solution {
    public int deleteAndEarn(int[] nums) {

        int max = 0;

        for (int num : nums) {
            max = Math.max(max, num);
        }

        int[] points = new int[max + 1];

        for (int num : nums) {
            points[num] += num;
        }

        int prev2 = 0;
        int prev1 = 0;

        for (int i = 0; i <= max; i++) {

            int current = Math.max(
                prev1,
                prev2 + points[i]
            );

            prev2 = prev1;
            prev1 = current;
        }

        return prev1;
    }
}
```

416. Partition Equal Subset Sum

Code:

```
class Solution {
    public boolean canPartition(int[] nums) {
        int sum = 0;
        for(int num : nums)
            sum+=num;
        if(sum%2!=0) return false;
        int target = sum/2;
        int n = nums.length;
        boolean[] dp = new boolean[target+1];
```

```

        if(nums[0]<=target)
            dp[nums[0]]=true;
        dp[0]=true;

        for(int i =1;i<n;i++){
            boolean[] curr = new boolean[target+1];
            curr[0] = true;
            for(int j =1;j<=target;j++){
                boolean np = dp[j];
                boolean p = false;
                if(nums[i]<=j){
                    p = dp[j-nums[i]];
                }
                curr[j]=p||np;
            }
            dp=curr;
        }

        return dp[target];
    }
}

```

494. Target Sum

Code:

```

class Solution {
    public int findTargetSumWays(int[] nums, int target) {
        int sum = 0;
        for (int num : nums)
            sum += num;
        if ((sum + target) % 2 != 0)
            return 0;
        target = Math.abs(target);
        int t = (sum + target) / 2;
        int[] dp = new int[t + 1];
        if (nums[0] <= t)
            dp[nums[0]] = 1;

        dp[0] = 1;

        if (nums[0] == 0)
            dp[0] = 2;

        for (int i = 1; i < nums.length; i++) {
            int[] curr = new int[t+1];
            for (int j = 0; j <= t; j++) {
                int np = dp[j];
                int p = 0;
                if (nums[i] <= j)

```

```

        p = dp[j - nums[i]];
        curr[j] = p + np;
    }
    dp=curr;
}
return dp[t];
}
}

```

983. Minimum Cost For Tickets

Code:

```

class Solution {
    public int mincostTickets(int[] days, int[] costs) {

        boolean[] travel = new boolean[366];
        for (int day : days) {
            travel[day] = true;
        }

        int[] dp = new int[366];

        for (int day = 1; day <= 365; day++) {
            if (!travel[day]) {
                dp[day] = dp[day - 1];
            }
            else {

                int oneDay = dp[day - 1] + costs[0];

                int sevenDay = dp[Math.max(0, day - 7)]
                    + costs[1];

                int thirtyDay = dp[Math.max(0, day - 30)]
                    + costs[2];

                dp[day] = Math.min(
                    oneDay,
                    Math.min(sevenDay, thirtyDay)
                );
            }
        }

        return dp[365];
    }
}

```

455 .

Code:

```
import java.util.*;

public class Main{
    public static void main(String[] args){
        Scanner in = new Scanner(System.in);
        int n = in.nextInt();
        int[] a = new int[n];
        HashMap<Integer,Integer> freq = new HashMap<>();
        int max =0;
        for(int i =0;i<n;i++){
            a[i]=in.nextInt();
            freq.put(a[i],freq.getDefault(a[i],0)+1);
            max = Math.max(max,a[i]);
        }

        long dp[] = new long[max+1];
        dp[0]=0;
        if (max >= 1) {
            dp[1] = (long)1 * freq.getDefault(1, 0);
        }
        for(int i =2 ;i<=max;i++){
            long take = (long)i*freq.getDefault(i,0)+dp[i-2];
            long notTake = dp[i-1];
            dp[i]=Math.max(take,notTake);
        }
        System.out.println(dp[max]);
    }
}
```